

16

Logic Programming Languages

Türkçe Anlatım • Terimler İngilizce

Predicate Calculus • Resolution • Unification

Prolog’a Özel Odak

Facts • Rules • Goals • Backtracking • Arithmetic

Review Questions (25) • Problem Set (10) • Programming Exercises

Gebze Technical University

CSE 341 — Programming Languages • Spring 2026

Sebesta, *Concepts of Programming Languages*, 12th Ed.

İçindekiler

1 Giriş: Neden Logic Programming?	4
1.1 Paradigma Karşılaştırması	4
1.2 Logic Programming'in Gücü ve Kısıtları	4
2 Predicate Calculus'a Kısa Giriş	5
2.1 Temel Yapı Taşları	5
2.2 Logical Connectives	5
2.3 Quantifiers	5
2.4 Clausal Form	6
3 Teorem İspat: Resolution ve Unification	6
3.1 Resolution İlkesi	6
3.2 Unification	7
3.3 Proof by Refutation	7
4 Logic Programming'e Genel Bakış	7
4.1 Horn Clause	7
4.2 Logic Program'ın Yapısı	8
5 Prolog'un Kökeni	8
6 Prolog'un Temel Unsurları	8
6.1 Terms: Atomlar, Değişkenler ve Compound Termler	8
6.2 Fact Statements	9
6.3 Rule Statements	10
6.4 Goal Statements	11
7 Prolog'un Çıkarım Mekanizması	12
7.1 Depth-First Arama ve Backtracking	12
7.2 Tam Çıkarım Örneği: ancestor Sorgusu	12
7.3 Backtracking ile Birden Fazla Çözüm	13
7.4 Arama Ağacı (Search Tree)	14
7.5 trace ile Adım Adım İzleme	14
8 Basit Aritmetik	15
8.1 is Operatörü	15
8.2 Karşılaştırma Operatörleri	15
8.3 Birleşik Örnek: Sayı Özellikleri	16
9 Özet: Temel Kavramlar Tablosu	18
10 Sınav İçin Kritik Noktalar	18

11 Review Questions — Soru Çözümleri	19
12 Problem Set — Soru Çözümleri	24
13 Programming Exercises — Çözümler	26

1 Giriş: Neden Logic Programming?

1.1 Paradigma Karşılaştırması

Bu noktaya kadar iki programlama stilini gördük: **imperatif** (nasıl yapılır, adım adım tarif) ve **fonksiyonel** (ne hesaplanır, matematiksel ifadeler). **Logic programming** (mantık programlama), bunlardan kökten farklıdır:

Logic Programming Paradigması

Logic programming’de programcı ne *nasıl yapılacağını*, ne de *hesaplanacağı* tarif eder. Bunun yerine:

- Domain hakkında bilinen **olgular (facts)** bildirir.
- Bu olgular arasındaki **ilişkileri (rules)** tanımlar.
- Sisteme bir **hedef (goal)** verir; sistem bu hedefe ulaşmaya çalışır.

Program = bilgi tabanı. Yürütme = otomatik teorem ispat.

Boyut	Imperatif	Fonksiyonel	Logic
Temel soru	<i>Nasıl?</i>	<i>Ne hesaplanır?</i>	<i>Ne doğrudur?</i>
Temel öge	Deyim	Fonksiyon	Önerme
Kontrol	Programcı	Kısmen programcı	Sistem (inference)
Yan etki	Var	Genellikle yok	Yok (saf versiyonda)
Örnek dil	C, Java, Python	Scheme, Haskell	Prolog

Imperatif: “Listeyi sırala: $i=0$, karşılaştır, yer değiştir, $i++$, tekrar...”

Fonksiyonel: `sort(list) = pivot al, küçükleri sırala, ekle, büyükleri sırala`

Logic: “Sıralı bir permütasyonu bul” — sisteme *ne istediğini* söylüyorsunuz, *nasıl bulacağını* değil.

1.2 Logic Programming’in Gücü ve Kısıtları

1981–1992 yılları arasında Japonya, “beşinci nesil bilgisayarlar” geliştirmek için devlet destekli büyük bir proje başlattı. Bu projede temel dil olarak **Prolog** seçildi. Paralel işleme ve yapay zeka hedeflendi. Proje nihai hedeflerine ulaşamadı, ancak Prolog’a olan ilgiyi ve araştırmaları büyük ölçüde artırdı; böylece dil akademide standart hale geldi.

Logic programming’in güçlü olduğu alanlar: **yapay zeka** (bilgi temsili, uzman sistemler), **doğal dil işleme** (dilbilgisi çözümleme), **veritabanı sorguları**, **otomatik teorem ispat**. Sayısal hesaplama, grafik işleme ve genel amaçlı uygulama geliştirme için *uygun değildir*.

2 Predicate Calculus'a Kısa Giriş

2.1 Temel Yapı Taşları

Predicate Calculus Terimleri

Object (Nesne): Hakkında önermeler kurulan varlık. Sabit (`john`), değişken (`X`) veya bileşik (`car(john)`) olabilir.

Proposition (Önerme): Doğru ya da yanlış olduğu belirlenebilen mantıksal cümle. Örnek: "John bir öğrencidir."

Atom (Atomic proposition): Mantıksal bağlaç içermeyen, bölünemez en küçük önerme birimi. Örnek: `likes(john, mary)`.

Predicate (Yüklem): Bir ya da daha fazla nesne hakkında özellik veya ilişki bildiren sembol. Örnek: `likes/2` (iki argümanlı), `mortal/1` (bir argümanlı).

Term: Sabit, değişken veya bileşik terim. Prolog'da her şey bir term'dir.

2.2 Logical Connectives

Sembol	Ad	Anlam	Prolog Karşılığı
$\neg P$	Negation	P değil	<code>not(P)</code>
$P \wedge Q$	Conjunction	P ve Q	<code>P, Q</code> (virgül)
$P \vee Q$	Disjunction	P veya Q	<code>P ; Q</code> (noktalı virgül)
$P \Rightarrow Q$	Implication	Eğer P ise Q	<code>Q :- P</code> (ters!)
$P \Leftrightarrow Q$	Equivalence	P ancak ve ancak Q	İki kural olarak yazılır

Matematikte $P \Rightarrow Q$ (P ise Q) iken, Prolog'da **sonuç solda, koşul sağda** yazılır:

`Q :- P.` anlamı: Q doğrudur, eğer P doğruysa.

`grandparent(X,Z) :- parent(X,Y), parent(Y,Z).`

okunuşu: "X, Z'nin büyükannesi/büyükbabasıdır; eğer X, Y'nin ebeveyni ve Y de Z'nin ebeveyni ise."

2.3 Quantifiers

Evrensel ve Varoluşsal Niceleyici

Universal quantifier $\forall$: "Tüm X için..." Örnek:

$$\forall X (\text{human}(X) \Rightarrow \text{mortal}(X))$$

Her insan ölümlüdür. Prolog'da **değişkenler evrensel olarak nicelendirilmiş kabul edilir.**

Existential quantifier $\exists$: "Öyle bir X vardır ki..." Örnek:

$$\exists X (\text{loves}(X, \text{mary}))$$

Mary'yi seven birisi vardır. Prolog'da bu tür ifadeler sorgulama ile ifade edilir.

2.4 Clausal Form

Clausal Form (Clause Biçimi)

Predicate calculus ifadelerinin mantık motorlarında işlenebilmesi için **clausal form**'a dönüştürülmesi gerekir. Clausal form, her satırın bir **disjunction of literals** (literal'lerin **VEYA'sı**) olduğu biçimdir.

Dönüşüm adımları:

1. $\Leftrightarrow$ ve $\Rightarrow$ 'yı temel bağlaçlara indirge.
2. $\neg$ operatörlerini atomlara taşı (De Morgan).
3. Existential quantifier'ları Skolem sabit/fonksiyonlarıyla ortadan kaldır.
4. $\forall$ quantifier'larını düşür; tüm değişkenler evrensel kabul edilir.
5. Conjunctive normal form'a getir.
6. Her clause'u ayrı bir satır olarak yaz.

Otomatik teorem ispatlayıcı (ve Prolog'un çekirdeği), yalnızca belirli bir formattaki cümleleri işleyebilir. Clausal form, resolution algoritmasının düzgün çalışabildiği standarttır. Prolog programcısı bunu doğrudan yazmaz – Prolog'un sözdizimi zaten clausal form'un bir altkümesidir.

3 Teorem İspat: Resolution ve Unification

3.1 Resolution İlkesi

Resolution

Resolution: İki clause'dan, bir atom hem pozitif hem negatif biçimde yer alıyorsa, bu atom eşleştirilerek iki clause'dan tek bir yeni clause türeten çıkarım kuralıdır. Robinson (1965) tarafından geliştirilmiştir.

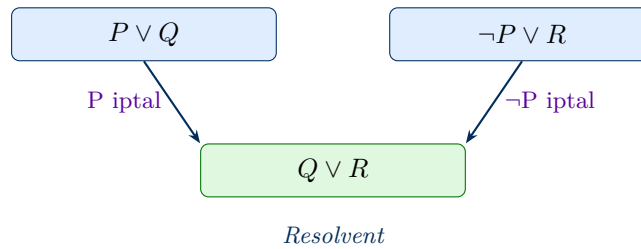
Eğer elimizde:

$$C_1 = P \vee Q \quad C_2 = \neg P \vee R$$

varsa, resolution şunu üretir:

$$\text{Resolvent} = Q \vee R$$

P ve $\neg P$ birbirini *çürütür*; kalanlar OR ile birleşir.



3.2 Unification

Unification (Birleştirme)

Unification: İki term'i eşit kılmak için değişkenlere değer atayan süreçtir. Substitution (yerine koyma) ile gerçekleşir.

Kurallar:

- Aynı atom kendiyle birleşir: $\text{john} = \text{john} \checkmark$
- Bir değişken herhangi bir term ile birleşir: $X = \text{john} \rightarrow X = \text{john}$
- İki compound term: functor ve arity eşleşmeli, her argüman özyinelemeli birleşmeli.
- Çevrimsel bağlama yasaktır (occurs check): $X = f(X)$ izin verilmez.

Term 1	Birleşir mi?	Substitution
john	john ile	--
X	john ile	$X = \text{john}$
parent(X, bob)	parent(tom, bob) ile	$X = \text{tom}$
likes(X, Y)	likes(john, mary) ile	$X=\text{john}, Y=\text{mary}$
f(X, g(X))	f(a, g(a)) ile	$X = a$
f(X)	g(X) ile	Hayir (farkli functor)
parent(X, X)	parent(tom, bob) ile	Hayir (X hem tom hem bob olamaz)

3.3 Proof by Refutation

Proof by Refutation — Çürütme ile İspat

Bir önermenin doğruluğunu ispatlamak için **negasyonunun yanlış** olduğunu gösteririz:

1. İspat edilmek istenen hedefin **negasyonunu** fact/rule tabanına ekle.
2. Resolution uygulayarak sadeleştir.
3. **Empty clause** (boş clause, $\square$) elde edilirse: çelişki bulundu.
4. Dolayısıyla negasyon yanlış, orijinal önerme **doğrudur**.

Prolog bir sorgu aldığı anda tam olarak bunu yapar: hedefi negasyona çevirip bilgi tabanındaki fact ve rule'larla resolution uygular. Empty clause elde ederse cevap **yes**. Hiç empty clause elde edemezse cevap **no**.

4 Logic Programming'e Genel Bakış

4.1 Horn Clause

Horn Clause

Horn clause: En fazla bir *pozitif literal* içeren bir clause'dur. İki özel biçimi vardır:

Fact (Olgu): Yalnızca bir pozitif literal.

Matematik: $\text{likes}(\text{bob}, \text{alice})$ Prolog: $\text{likes}(\text{bob}, \text{alice}).$

Rule (Kural): Bir pozitif literal *head* ve sıfır ya da daha fazla pozitif literal *body*.

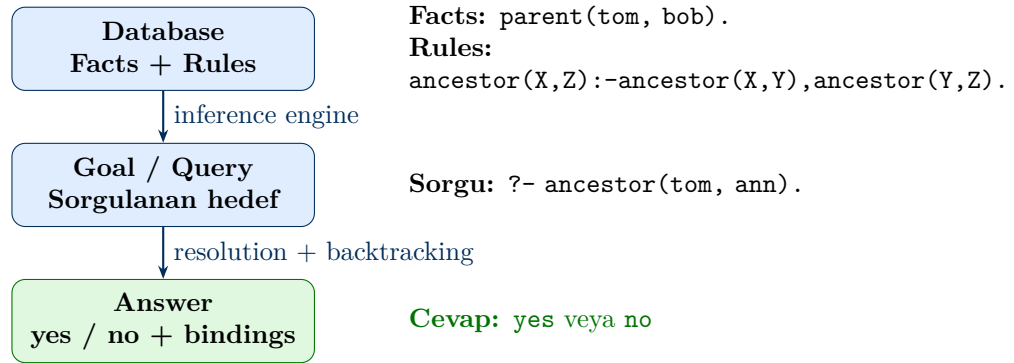
Matematik: $\text{grandparent}(X, Z) \leftarrow \text{parent}(X, Y) \wedge \text{parent}(Y, Z)$

Prolog: `grandparent(X,Z) :- parent(X,Y) , parent(Y,Z) .`

Prolog, tüm programı Horn clause biçiminde yazar. Clausal form dönüşümü programcıya bırakılmaz.

Genel predicate calculus, Horn clause biçiminden çok daha ifade gücüne sahiptir. Horn clause, gerçek anlamda bir *kısıtlamadır* – bazı mantıksal ilişkiler doğrudan Horn clause olarak ifade edilemez. Prolog bu kısıtlamayı kabul eder çünkü Horn clause üzerinde çalışan resolution **verimli** ve **belirleyici** (deterministic yönde) uygulanabilir.

4.2 Logic Program'ın Yapısı



5 Prolog'un Kökeni

Prolog (PROgramming in LOGic), 1972'de Fransa'da Marsilya Üniversitesi'nde **Alain Colmerauer** ve ekibi tarafından geliştirildi. İlk uygulama doğal dil işleme problemleri için tasarlandı.

Eşzamanlı olarak, **Robert Kowalski** İngiltere'de Edinburgh Üniversitesi'nde Horn clause'lar üzerinde resolution'ı teorik olarak çalıştı. Bu iki grubun çalışmaları birleşti.

1977'de **David Warren** Edinburgh'da **DEC-10 Prolog**'u geliştirdi ve bu, modern Prolog'un asıl referans implementasyonu oldu. Warren Abstract Machine (WAM), bugün hâlâ Prolog implementasyonlarının teorik temeli olarak kullanılmaktadır.

Günümüzde en yaygın kullanılan açık kaynak implementasyon **SWI-Prolog**'dur.

6 Prolog'un Temel Unsurları

6.1 Terms: Atomlar, Değişkenler ve Compound Termler

Prolog Term Türleri

Atom: Küçük harfle başlayan identifier, sayısal olmayan sabit, veya tek tırnak içindeki sembol.

- Örnekler: `john`, `apple`, `x`, `'New York'`, `likes`

Number: Tamsayı veya gerçek sayı.

- Örnekler: `42`, `3.14`, `-7`

Variable: Büyük harfle ya da alt çizgi (`_`) ile başlayan identifier. İlk bağlamadan sonra değiştirilemez (single-assignment).

- Örnekler: `X`, `Parent`, `Name`, `_` (anonim değişken)

Compound term (Structure): Bir *functor* ve parantez içinde virgülle ayrılmış argümanlar.

- Örnekler: `person(john, 30)`, `point(3, 4)`, `f(X, g(a))`
- Functor + arity birlikte bir **predicate** tanımlar: `person/2`, `point/2`

Prolog'da bir değişken bağlandıktan sonra **yeniden bağlanamaz**. `X = 5`, `X = 6` yazılamaz — ikinci `X = 6` başarısız olur çünkü `X` zaten 5'e bağlıdır.

Bu, imperatif dillerdeki atama deyiminden kökten farklıdır. `X = 5` Prolog'da atama değil, **unification (birleştirme)** işlemidir.

Ayrıca: Büyük/küçük harf ayrımı kritiktir. `john` bir atom, `John` bir değişkendir.

Listing 1: Prolog term örnekleri

```

1 % Atoms
2 john           % identifier atom
3 'New York'     % spaces allowed with quotes
4 red            % symbolic constant
5
6 % Variables
7 X, Y, Z        % standard variables
8 Parent         % meaningful variable name
9 _              % anonymous variable (wildcard)
10
11 % Compound terms
12 parent(tom, bob) % parent/2 predicate
13 likes(mary, X)   % one variable
14 date(2026, may, 18) % date structure
15 point(X, Y)      % 2D point
16 person(Name, Age, City) % person structure

```

6.2 Fact Statements

Fact (Olgu) Bildirimi

Fact, Prolog veri tabanına koşulsuz olarak eklenen bir **Horn clause head**'dir. Bir ilişkinin *her zaman doğru* olduğunu bildirir. Sözdizimi:

`predicate_name(arg1, arg2, ...)`.

Nokta (.) zorunludur — her Prolog ifadesi nokta ile biter.

Listing 2: Prolog fact örnekleri — aile ilişkileri

```

1 % Ebeveyn ilişkileri
2 parent(tom, bob). % tom, bob'un ebeveynidir
3 parent(tom, liz). % tom, liz'in ebeveynidir
4 parent(bob, ann). % bob, ann'in ebeveynidir
5 parent(bob, pat). % bob, pat'in ebeveynidir
6 parent(pat, jim). % pat, jim'in ebeveynidir
7
8 % Cinsiyet bilgisi
9 male(tom).
10 male(bob).
11 male(pat).

```

```

12 male(jim).
13 female(liz).
14 female(ann).
15
16 % Hobi bilgisi
17 likes(bob, tennis).
18 likes(liz, music).
19 likes(ann, tennis).
20
21 % Sayisal bilgi
22 age(tom, 60).
23 age(bob, 35).
24 age(ann, 10).

```

Fact'ler bir **bilgi tabanı (knowledge base)** oluşturur. Prolog'un **closed-world assumption (kapalı dünya varsayımı)** modeli şunu söyler: Bilgi tabanında olmayan her şey yanlıştır. Yani `likes(bob, chess)` bilgi tabanında yoksa, Prolog bunun kesinlikle yanlış olduğunu kabul eder.

6.3 Rule Statements

Rule (Kural) Bildirimi

Rule, bir bilginin başka bilgilere bağlı olarak ne zaman doğru olduğunu tanımlar. Biçimi: `head :- body.`

Head: Doğrulanmak istenen tek pozitif literal.

Body: Virgülle ayrılmış koşullar listesi (conjunction). Her virgül “ve” anlamına gelir.

Noktalı virgül (;): Disjunction (veya) anlamına gelir.

Okunuşu: “head doğrudur, eğer body'deki tüm koşullar doğruysa.”

Listing 3: Prolog rule örnekleri — aile ilişkileri

```

1 % Buyukanne/buyukbaba
2 grandparent(X, Z) :-
3     parent(X, Y),          % X, Y'nin ebeveyni
4     parent(Y, Z).          % Y de Z'nin ebeveyni
5
6 % Kardes (ayni ebeveyni paylasiyor)
7 sibling(X, Y) :-
8     parent(Z, X),
9     parent(Z, Y),
10    X \= Y.                  % X ve Y farkli olmalı!
11
12 % Ata/Ataturk (ancestor) - ozinelemeli kural
13 ancestor(X, Y) :- parent(X, Y).
14 ancestor(X, Y) :-
15     parent(X, Z),
16     ancestor(Z, Y).
17
18 % Amca/teyze
19 uncle(X, Y) :-
20     sibling(X, Z),
21     parent(Z, Y),
22     male(X).
23

```

```

24 % Buyukanne
25 grandmother(X, Y) :-
26     grandparent(X, Y),
27     female(X).
28
29 % Var mi? - query icin
30 has_grandchild(X) :-
31     grandparent(X, _). % _ = anonim degisken, degeri onemsiz

```

- **Virgül = AND, noktalı virgül = OR:** a, b = “a ve b”, $a ; b$ = “a veya b”.
- **\= operatörü:** Birleşeme testi. $X \neq Y$ yalnızca X ve Y birleşemiyorsa doğrudur.
- **Kuralların sırası önemlidir:** Prolog kuralları *yukarıdan aşağıya* dener. Özyinelemeli kurallarda base case genellikle önce gelmelidir.
- **Body’deki sıra önemlidir:** Prolog body’deki koşulları *soldan sağa* işler. Bu hem verimlilik hem doğruluk açısından kritiktir.

6.4 Goal Statements

Goal (Hedef) — Sorgu Modu

Prolog yorumlayıcısına soru sormak için **goal statement** kullanılır.

Sözdizimi: `?- goal.` (soru işareti ve tire)

Üç tür sonuç:

- **yes / true:** Hedef bilgi tabanında ispat edilebildi.
- **no / false:** Hedef ispat edilemedi (closed-world assumption).
- **Değişken bağlamaları:** Sorgu değişken içeriyorsa, sistemi doğrulayan atamalar listelenir.

Listing 4: Prolog goal/sorgu örnekleri

```

1 % Basit sorgular
2 ?- parent(tom, bob).           % Cevap: yes
3 ?- parent(tom, ann).          % Cevap: no (ann'in ebeveyni bob)
4 ?- grandparent(tom, ann).     % Cevap: yes (tom->bob->ann)
5
6 % Degiskenli sorgular - kim?
7 ?- parent(tom, X).
8 % X = bob ;
9 % X = liz .
10
11 ?- parent(X, ann).
12 % X = bob .
13
14 % Çok degiskenli sorgular
15 ?- parent(X, Y).
16 % X = tom, Y = bob ;
17 % X = tom, Y = liz ;
18 % X = bob, Y = ann ;
19 % ... (butun ciftler)
20
21 % Birlesik sorgu - AND
22 ?- parent(tom, X), parent(X, Y).
23 % X = bob, Y = ann ;
24 % X = bob, Y = pat .
25
26 % Buyukanne sorgular

```

```

27 ?- grandparent(X, ann).      % X = tom
28 ?- has_grandchild(tom).      % yes

```

Prolog bir sorguya ilk cevabı bulduğunda **durur** ve kullanıcıya gösterir. Kullanıcı ; (noktalı virgül) yazarsa Prolog **backtrack** ederek bir sonraki çözümü arar. . yazılırsa (veya Enter) sorgulama biter.

7 Prolog'un Çıkarım Mekanizması

7.1 Depth-First Arama ve Backtracking

Prolog'un Inferencing Stratejisi

Prolog, bir hedefi ispatlamaya çalışırken şu stratejiyi uygular:

1. Veritabanını **yukarıdan aşağıya** tara; hedefle **unify** olan ilk clause'u bul.
 2. Bu clause'un body'sini işle — her bir body literal'ını özyinelemeli olarak ispat et.
 3. Body literal'ı başarısız olursa **backtrack** et: bir önceki seçim noktasına dön, farklı bir clause dene.
 4. **Empty clause** elde edilirse **success** (başarı).
 5. Hiç seçenek kalmazsa **failure** (başarısızlık) — cevap no.
- Bu strateji **depth-first search** (derinlik öncelikli arama) + **backtracking**'dir.

Prolog'un depth-first stratejisi verimlidir ama **eksiksiz (complete) değildir**. Eğer bilgi tabanında sonsuz bir dal varsa (ör. yanlış yazılmış özyinelemeli kural), Prolog o dalda sonsuza kadar ilerleyebilir — tüm çözümleri bulamaz. Bu nedenle kuralların yazılış sırası kritiktir.

7.2 Tam Çıkarım Örneği: ancestor Sorgusu

Veri tabanı:

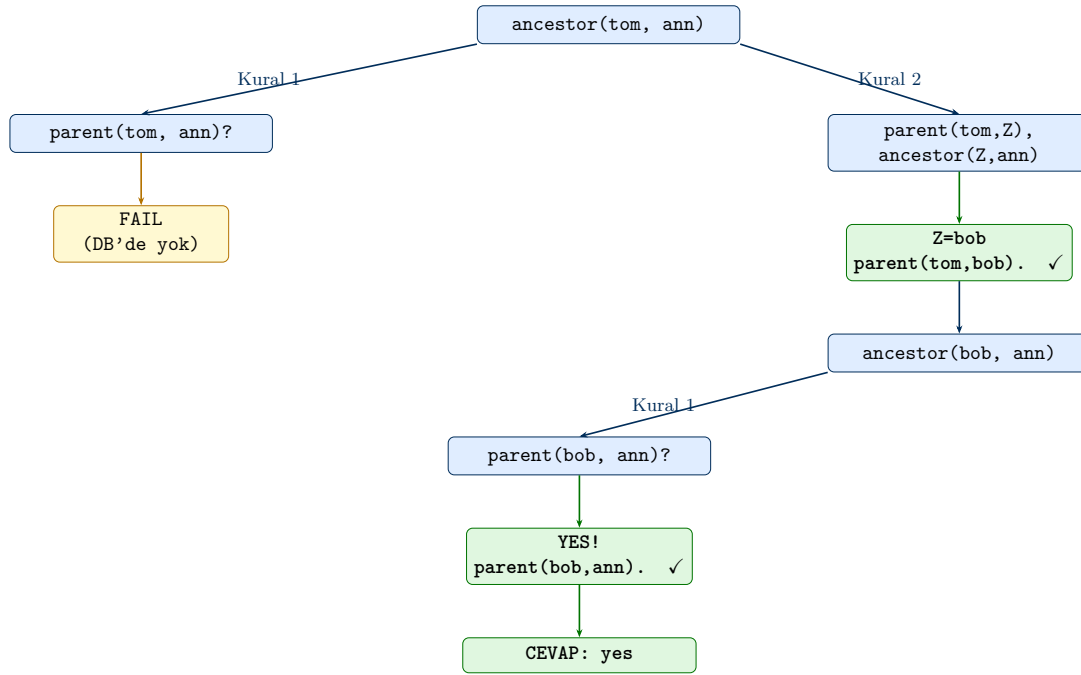
Listing 5: Veritabanı + sorgu

```

1 % Facts
2 parent(tom, bob).
3 parent(bob, ann).
4
5 % Rules (siralama önemli!)
6 ancestor(X, Y) :- parent(X, Y).      % Kural 1
7 ancestor(X, Y) :-                    % Kural 2
8     parent(X, Z), ancestor(Z, Y).
9
10 % Sorgu
11 ?- ancestor(tom, ann).

```

Çözüm izi (trace):



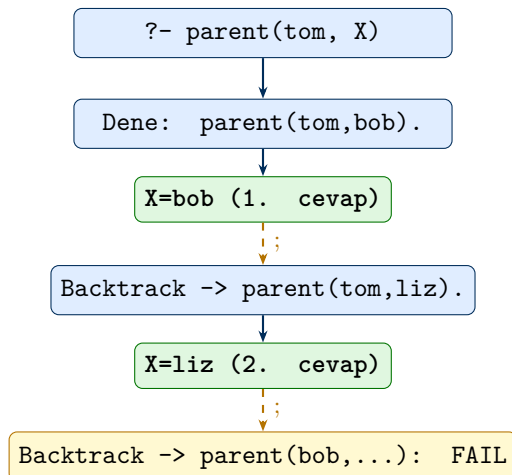
7.3 Backtracking ile Birden Fazla Çözüm

Listing 6: parent(tom,X) – backtracking ile tum cevaplar

```

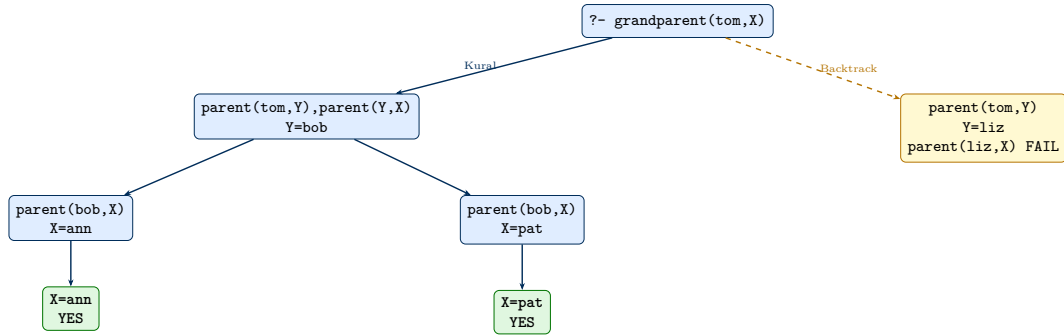
1 % Veritabani:
2 % parent(tom, bob).
3 % parent(tom, liz).
4 % parent(bob, ann).
5
6 ?- parent(tom, X).
7 % Adim 1: parent(tom, bob) ile eslesiyor -> X = bob
8 %       Kullanici ; yazarsa...
9 % Adim 2: parent(tom, liz) ile eslesiyor -> X = liz
10 %       Kullanici ; yazarsa...
11 % Adim 3: parent(bob, ann) - tom yok, bitmedi;
12 %       parent(pat, jim) - tom yok; ... sonuc:
13 % false.
14
15 % Cevaplar sirasiyla:
16 % X = bob ;
17 % X = liz .

```



7.4 Arama Ağacı (Search Tree)

Daha karmaşık sorguların çözüm sürecini **arama ağacı** (search/resolution tree) ile görselleştirmek mümkündür. Prolog bu ağacı **depth-first, left-to-right** dolaşır.



7.5 trace ile Adım Adım İzleme

Prolog'un çoğu implementasyonunda **trace** komutuyla yorumlayıcının attığı her adım izlenebilir. Dört port modeli:

Dört Port Modeli (Box Model)

Her Prolog hedefi bir kutu gibi düşünülür ve dört “kapı” üzerinden geçiş yapar:

- **CALL:** Hedef ilk kez deniyor.
- **EXIT:** Hedef başarılı biçimde ispat edildi.
- **REDO:** Kullanıcı ; yazarak yeni çözüm istiyor; backtrack.
- **FAIL:** Hedef ispat edilemedi; backtrack.

Listing 7: trace komutu ile çıktı

```

1  ?- trace, parent(tom, X).
2
3  CALL: parent(tom, _G1)
4  EXIT: parent(tom, bob)      % X = bob
5
6  X = bob ;                  % kullanıcı ; yazdı
7
8  REDO: parent(tom, _G1)
9  EXIT: parent(tom, liz)     % X = liz
10
11 X = liz ;                  % kullanıcı tekrar ; yazdı
12
13 REDO: parent(tom, _G1)
14 FAIL: parent(tom, _G1)     % başka eşleşme yok
15 false.
16
17 ?- notrace.                % trace'i kapat

```

8 Basit Aritmetik

8.1 is Operatörü

is Operatörü — Aritmetik Değerlendirme

Prolog'da normal birleştirme (=) aritmetik ifadeleri **değerlendirmez**. Aritmetik hesaplama için **is** operatörü kullanılır:

Variable is ArithmeticExpression

is, sağ taraftaki aritmetik ifadeyi önce **hesaplar (evaluates)**, sonra sol taraftaki değişkenle birleştirir. Sol taraf ya serbest değişken ya da aynı değere sahip sabit olmalıdır.

- $X = 3 + 5 \rightarrow X$, yapısal term $+(3,5)$ olur; *hesaplamaz*, $X = 8$ değildir.
- $X \text{ is } 3 + 5 \rightarrow X$, hesaplanmış değer 8 olur; $X = 8$.
- $8 \text{ is } 3 + 5 \rightarrow \text{true}$ (sol taraf sayıysa kontrol yapar).
- $3 + 5 \text{ is } 8 \rightarrow \text{HATA}$ (sol taraf hesaplanamaz).

Listing 8: is ile aritmetik örnekleri

```

1 % Basit hesaplama
2 ?- X is 3 + 5.           % X = 8
3 ?- X is 10 * 4 + 2.      % X = 42
4 ?- X is 2 ** 10.         % X = 1024.0
5 ?- X is sqrt(144).       % X = 12.0
6
7 % Karsilastirma - = degil is kullan
8 ?- 8 is 3 + 5.          % true
9 ?- X = 3 + 5.           % X = 3+5 (yapisal term, hesaplanmadi!)
10
11 % Degisken sol tarafa olmalı
12 ?- Y = 10, X is Y * 3.  % X = 30
13 ?- X is Y + 1.          % HATA - Y unbound (bagi yok)
14
15 % Aritmetik fonksiyonlar
16 ?- X is max(5, 9).      % X = 9
17 ?- X is min(5, 9).      % X = 5
18 ?- X is abs(-7).        % X = 7
19 ?- X is 17 mod 5.        % X = 2
20 ?- X is 17 div 5.        % X = 3 (integer division)

```

8.2 Karşılaştırma Operatörleri

Aritmetik Karşılaştırma Operatörleri

Bu operatörler, iki taraftaki aritmetik ifadeyi önce değerlendirip sonra karşılaştırır. Bunlar serbest değişken içeremez.

Operatör	Anlam	Örnek
$X \text{ } := \text{ } Y$	Aritmetik eşitlik	$3+5 \text{ } := \text{ } 8 \rightarrow \text{true}$
$X \text{ } \backslash= \text{ } Y$	Aritmetik eşitsizlik	$3 \text{ } \backslash= \text{ } 4 \rightarrow \text{true}$
$X < Y$	Küçüktür	$3 < 5 \rightarrow \text{true}$
$X > Y$	Büyüktür	$9 > 4 \rightarrow \text{true}$
$X \text{ } = < \text{ } Y$	Küçük eşit	$5 \text{ } = < \text{ } 5 \rightarrow \text{true}$
$X \text{ } \geq \text{ } Y$	Büyük eşit	$7 \geq 3 \rightarrow \text{true}$

- `==` ile `=` karıştırılmamalıdır: `3+5 = 8` **false** (yapısal eşitsizlik), `3+5 == 8` **true** (aritmetik eşitlik).
- `=<` (küçük eşit) `<=` değildir! C/Java alışkanlığıyla `<=` yazmak sözdizimi hatasıdır.
- Değişken içeren karşılaştırma: `X < 5` sorgulanırken `X` bağlı olmalı, aksi hâlde hata.

Listing 9: Karsilastirma ve aritmetik birlikte

```

1  % Mutlak deger
2  myabs(X, X) :- X >= 0.      % X pozitif veya sifirsa
3  myabs(X, Y) :- X < 0, Y is -X.
4
5  % Iki sayinin buyugu
6  mymax(X, Y, X) :- X >= Y.
7  mymax(X, Y, Y) :- Y > X.
8
9  % Kare alma
10 square(X, Y) :- Y is X * X.
11
12 % Factorial - aritmetik ile
13 factorial(0, 1).
14 factorial(N, F) :-
15     N > 0,
16     N1 is N - 1,
17     factorial(N1, F1),
18     F is N * F1.
19
20 % Fibonacci
21 fib(0, 0).
22 fib(1, 1).
23 fib(N, F) :-
24     N > 1,
25     N1 is N - 1,
26     N2 is N - 2,
27     fib(N1, F1),
28     fib(N2, F2),
29     F is F1 + F2.
30
31 % Alan hesabi
32 circle_area(R, A) :- A is 3.14159 * R * R.
33
34 % Toplam (accumulator ile - tail-recursive)
35 sum_to(0, 0).
36 sum_to(N, S) :-
37     N > 0,
38     N1 is N - 1,
39     sum_to(N1, S1),
40     S is S1 + N.
41
42 % Sorgu ornekleri
43 ?- factorial(5, F).      % F = 120
44 ?- fib(8, F).           % F = 21
45 ?- mymax(7, 3, M).      % M = 7
46 ?- sum_to(10, S).       % S = 55

```

8.3 Birleşik Örnek: Sayı Özellikleri

Listing 10: Sayi teorisi sorulari

```

1  % Tek sayi mi?
2  odd(N) :- N mod 2 == 1.
3  even(N) :- N mod 2 == 0.
4
5  % Asallik testi - naive yontem
6  prime(2).
7  prime(3).
8  prime(N) :-
9      N > 3,
10     N mod 2 \= 0,          % cift sayilar asil degil
11     \+ has_factor(N, 3).
12
13 has_factor(N, F) :-
14     N mod F == 0.
15 has_factor(N, F) :-
16     F * F < N,
17     F2 is F + 2,
18     has_factor(N, F2).
19
20 % Bolunebilirlik
21 divides(D, N) :- N mod D == 0.
22
23 % Bolenlerin sayisi
24 count_divisors(N, Count) :-
25     findall(D, (between(1, N, D), divides(D, N)), Ds),
26     length(Ds, Count).
27
28 % Ornek sorgular
29 ?- odd(7).           % true
30 ?- even(6).          % true
31 ?- prime(17).        % true
32 ?- prime(15).        % false
33 ?- divides(3, 12).   % true

```

9 Özet: Temel Kavramlar Tablosu

Kavram	Açıklama	Prolog Karşılığı
Atom	Küçük harfle başlayan sabit	john, apple
Variable	Büyük harfle başlayan, bağlanabilir	X, Parent
Compound term	functor + argümanlar	person(john, 30)
Fact	Koşulsuz olgu	parent(tom,bob).
Rule	Koşullu ilişki	head :- body.
Goal	Sorgu / ispat edilecek hedef	?- ancestor(X,ann).
Unification	İki term'i eşit kılma (=)	X = john
Resolution	Clause'lardan yeni clause türetme	Otomatik (inference engine)
Backtracking	Başarısızlıkta önceki seçeneğe dönme	Otomatik
is	Aritmetik değerlendirme	X is 3 + 5
==	Aritmetik eşitlik	3+5 == 8
\=	Birleşememe testi	X \= Y
Horn clause	En fazla 1 pozitif literal	Prolog'un temel yapısı
Depth-first search	Prolog'un arama stratejisi	Sıra önemlidir!
Closed-world assumption	Veritabanında olmayan = yanlış	no cevabı
Proof by refutation	Negasyonu çürüterek ispat	Prolog motoru bunu yapar
Trace	Adım adım çıkarım izleme	trace. komutu

Operator	Ne Yapar?	Dikkat
=	Unification – yapısal eşleştirme	3+5 = 8 false!
is	Aritmetik değerlendir + birleştir	Sol taraf bağlı olmamalı
==	Her iki tarafı hesapla, karşılaştır	3+5 == 8 true
==	Değerlendirmeden yapısal eşitlik	X == X true
\=	Unification başarısız mı?	X \= Y: fark testi
\==	Yapısal eşit değil mi?	a \== b true
=<	Küçük eşit	C'deki <= değil!

10 Sınav İçin Kritik Noktalar

- Horn clause nedir?** En fazla 1 pozitif literal içeren clause. Prolog'un temel yapısı.
- Prolog'da :- ne anlama gelir?** “Eğer” (implication, ters yönlü). Head doğrudur, body doğruysa.
- Unification ile aritmetik değerlendirme farkı:** = yapısal, is aritmetik. X = 3+5 ile X is 3+5 farklı!
- Prolog'un arama stratejisi:** Depth-first + backtracking. Yukarıdan aşağıya, soldan sağa.
- Closed-world assumption:** Bilgi tabanında olmayan her şey yanlıştır.
- Proof by refutation:** Hedefin negasyonu eklenir, empty clause elde edilirse hedef doğru.
- Resolution nedir?** İki clause'u birleştirerek yeni clause türeten çıkarım kuralı.

8. **Unification nedir?** İki term'i eşit kılmak için değişkenlere değer atayan süreç.
9. **Kural sırası önemlidir:** Base case önce yazılmalıdır; aksi hâlde sonsuz döngü riski.
10. **Değişkenler single-assignment:** $X = 5$, $X = 6$ çelişki, **başarısız** olur.
11. **Anonim değişken _:** Her yerde farklı bir değişken gibi davranır; değeri önemsizdir.
12. $==$ vs $=:$ $==$ iki tarafı *hesaplar* sonra karşılaştırır; $=$ yalnızca birleştirir.
13. $=<$ ile $<=$: Prolog'da küçük-eşit $=<$ 'tür; $<=$ sözdizimi hatasıdır.
14. **Prolog tamamlanabilir değil:** Depth-first + yanlış kural sırası sonsuz döngüye yol açar.
15. **Prolog 1972 Colmerauer:** Edinburgh'da Warren'ın DEC-10 Prolog'u referans uygulamadır.

11 Review Questions — Soru Çözümleri

S1. What are the three components of a Prolog database?

Cevap

Prolog veri tabanının üç bileşeni:

1. **Facts (Olgular):** Koşulsuz, doğru kabul edilen atomik önermeler. Örnek: `parent(tom, bob)`.
2. **Rules (Kurallar):** Koşullu ilişkiler; head body şeklinde. Örnek: `grandparent(X,Z) :- parent(X,Y), parent(Y,Z)`.
3. **Goals/Queries (Hedefler):** Kullanıcının sorgulayıp ispat ettirmek istediği önermeler. Örnek: `?- grandparent(tom, ann)`.

S2. What is a fact statement in Prolog?

Cevap

Prolog'da **fact statement**, bilgi tabanına koşulsuz olarak eklenen Horn clause head'dir. Bir ilişkinin veya özelliğin *her zaman doğru* olduğunu bildirir. Sözdizimi: `predicate(arg1, arg2, ...)`. Nokta zorunludur. Faktlar, resolution için başlangıç noktasıdır; değişken içerebilir.

S3. What is a rule statement in Prolog?

Cevap

Rule statement, bir bilginin başka bilgilere bağlı olarak ne zaman doğru olduğunu tanımlayan Horn clause'dur. Biçimi: `head :- body`. Baş (head) tek bir pozitif literal, gövde (body) virgülle ayrılmış koşullar listesidir. Okunuşu: "Head doğrudur, eğer body'deki tüm koşullar doğruysa." Kurallar özyinelemeli olabilir.

S4. What kind of search strategy does Prolog use?

Cevap

Prolog, **depth-first search (derinlik öncelikli arama)** stratejisini **backtracking** ile birlikte kullanır. Kurallar yukarıdan aşağıya, body koşulları soldan sağa işlenir. Bir yol

başarısız olduğunda en son seçim noktasına geri dönerek alternatif denenir. Bu strateji, tüm çözümleri garantilemez (sonsuz dallarda takılabilir) ancak verimlidir.

S5. What is the closed-world assumption in Prolog?

Cevap

Closed-world assumption (kapalı dünya varsayımı): Prolog veri tabanında yer almayan her önerme *yanlış* kabul edilir. Yani Prolog, bilmediği şeyi “bilinmiyor” olarak değil, “yanlış” olarak yorumlar. Örneğin `likes(bob, chess)` bilgi tabanında yoksa `?- likes(bob, chess).` sorgusuna cevap `no`'dur; bu “kesinlikle yanlış” anlamına gelir.

Bu varsayım gerçek dünya modellemesinde sorunludur: bir şeyin bilinmemesi onun yanlış olduğu anlamına gelmez.

S6. What is resolution in logic programming?

Cevap

Resolution: Robinson (1965) tarafından geliştirilen, iki clause'dan yeni bir clause türeten çıkarım kuralıdır. Bir atom hem pozitif hem negatif biçimde iki farklı clause'da yer alıyorsa, bu atom *çürütülerek* (cancel) iki clause birleştirilir. Eğer sonuç **empty clause (boş clause)** ise çelişki bulunmuş, hedef ispatlanmış demektir. Prolog motoru resolution'ı otomatik olarak uygular.

S7. What is unification?

Cevap

Unification (birleştirme): Değişkenlere değer atayarak iki term'i eşit kılma işlemidir. Prolog'da = operatörü unification gerçekleştirir. Aynı atomlar her zaman birleşir. Bir değişken herhangi bir term ile birleşebilir (o değişkene bağlanır). Compound term'ler, functor ve arity eşleşiyorsa, argümanlar özyinelemeli olarak birleştirilir. Prolog'un çalışma mekanizmasının temelidir.

S8. What is the difference between = and == in Prolog?

Cevap

- = (Unification): İki term'i birleştirmeye *çalışır*. Değişken bağlaması yapar. `X = 5` başarılıdır; X, 5'e bağlanır.
- == (Structural equality): İki term'in *şu anda* yapısal olarak eşit olup olmadığını değerlendirilmeden kontrol eder. Değişken bağlaması yapmaz. `X == 5` — X zaten 5'e bağlı değilse **false**'tur.

Özet: = bağlar, == yalnızca kontrol eder.

S9. What is the difference between = and is in Prolog?

Cevap

- = (Unification): Aritmetik ifadeyi *hesaplamaz*. `X = 3 + 5` sonucu: X, yapısal term `+(3,5)` olur; X'in değeri 8 değildir.
- is (Arithmetic evaluation): Sağ taraftaki aritmetik ifadeyi *önce hesaplar*, sonra sol taraftaki değişkenle birleştirir. `X is 3 + 5` sonucu: `X = 8`.

S10. What is the neck (:-) symbol in Prolog?

Cevap

Neck ($:-$) sembolü, Prolog'da rule (kural) tanımlamak için kullanılır. Sol tarafındaki **head** (sonuç), sağ tarafındaki **body** (koşullar) doğru olduğunda ispatlanabilir. Matematiksel anlamı: ters yönlü implication ($\leftarrow$). Okunuşu: "...eğer ...ise" (if). Örnek: `grandparent(X,Z) :- parent(X,Y), parent(Y,Z).`

S11. What is a compound term in Prolog?

Cevap

Compound term (bileşik terim): Bir **functor** (atom) ve parantez içinde virgülle ayrılmış argümanlar listesinden oluşan yapıdır. Argümanlar da term'dir (atom, sayı, değişken veya başka bir compound term). Functor ve argüman sayısı (arity) birlikte o compound term'in *predicate*'ini belirler. Örnek: `point(3, 4)` functor=`point`, arity=2.

S12. What is an atom in Prolog?

Cevap

Prolog'da **atom**, küçük harfle başlayan identifier (`john`, `apple`), operatör sembolü (`+`, `:-`) veya tek tırnak içindeki her türlü karakter dizisidir (`'New York'`). Sayılar atom *değildir*; ayrı bir term kategorisidir. Atomlar, compound term'lerin functor'ı olarak veya tek başına sabit olarak kullanılır.

S13. What is a variable in Prolog?

Cevap

Prolog'da **variable**, büyük harfle veya `_` ile başlayan tanımlayıcıdır. Değişkenler **single-assignment**: bir kez bir term'e bağlandıktan sonra aynı ispat yolu boyunca değiştirilemez. Backtracking sırasında bağlama serbest bırakılır. `_` anonim değişkendir; her geçişi bağımsız ve değeri önemsiz kabul edilir.

S14. How does Prolog prove a goal?

Cevap

Prolog, **proof by refutation (çürütme ile ispat)** yöntemini kullanır: hedefin negasyonunu veritabanına ekler ve resolution uygular. **Empty clause** elde edilirse negasyon çelişkidir, dolayısıyla orijinal hedef **doğrudur**. Pratikte: veri tabanını yukarıdan aşağıya tarar, hedefle unify olan ilk clause'u bulur, body'yi özyinelemeli olarak ispat eder, başarısızlıkta backtrack eder.

S15. What is backtracking in Prolog?

Cevap

Backtracking: Bir ispat yolunda başarısızlıkla karşılaşıldığında, en son seçim noktasına geri dönerek henüz denenmemiş alternatif clause'u seçme sürecidir. Backtracking, o seçim noktasında yapılan tüm değişken bağlamalarını *geri alır* (undo). Böylece Prolog sistematik olarak tüm olası çözüm yollarını keşfedebilir. Kullanıcı ; yazarak ek çözüm istediğinde de backtracking tetiklenir.

S16. What is the occurs check in unification?

Cevap

Occurs check: Bir değişkenin, o değişkeni içeren bir term ile birleştirilip birleştirilmediğini kontrol eden mekanizmadır. Örneğin $X = f(X)$ ifadesi sonsuz bir term ($f(f(f(\dots)))$) oluşturur ve mantıksal açıdan geçersizdir. Teorik açıdan tam doğru unification, bu occurs check'i her birleştirmede yapmalıdır. Ancak bu pahalı olduğundan, Prolog'un çoğu implementasyonu **occurs check yapmaz** — bu teknik olarak hatalı unification'a yol açabilir. `unify_with_occurs_check/2` ile açıkça kullanılabilir.

S17. Why does the order of rules in Prolog matter?

Cevap

Prolog depth-first arama yaptığından, kuralların yazıldığı sıra hem **doğruluğu** hem de **sonlanmayı** etkiler. Özyinelemeli bir predicate'te base case (temel durum) önce yazılmazsa, recursive case sonsuz döngüye girebilir. Ayrıca birden fazla çözüm varsa, sıra hangi çözümün önce bulunacağını belirler. Bu durum, Prolog'u diğer logic programming modellerinden (tam arama yapanlardan) ayıran önemli bir pratik kısıtlamadır.

S18. What is proof by refutation and why does Prolog use it?

Cevap

Proof by refutation: Bir önermenin doğruluğunu ispat etmek için *yanlış olduğunu varsayıp* çelişkiye ulaşma yöntemidir. Prolog bunu kullanır çünkü: (1) Verimlidir — resolution tek bir çıkarım kuralıdır; (2) Horn clause kümesi üzerinde *complete* (eksiksiz) strateji mümkündür; (3) Hem fact hem rule hem de query aynı formatta (Horn clause) ifade edilir, dolayısıyla uniform bir mekanizma yeterlidir.

S19. What is a Horn clause, and why is Prolog restricted to Horn clauses?

Cevap

Horn clause: En fazla bir *pozitif literal* içeren clause'dur. Prolog, Horn clause kısıtlamasını kabul eder çünkü: (1) Horn clause üzerinde resolution **belirleyici (deterministic)** biçimde uygulanabilir — hangi sırayla deneneceği önceden belirlenebilir; (2) Genel clausal form (birden fazla pozitif literal) üzerinde resolution yönetilemez derecede dal üretir; (3) Gerçek dünya uygulamalarında çoğu bilgi Horn clause olarak ifade edilebilir. Kısıtlamanın bedeli: Bazı mantıksal ilişkiler doğrudan ifade edilemez (özellikle negation as failure sorunları).

S20. What is the difference between predicate calculus and propositional logic?

Cevap

Propositional logic (önerme mantığı): Yalnızca tam önermelerle çalışır; nesneler ve özellikler hakkında ayrıntılı ifade kurulamaz. Örnek: $P = \text{"Tom bir insan"}$. P doğru ya da yanlıştır ama kim olduğu ifade edilemez.

Predicate calculus (yüklem hesabı / first-order logic): Nesneler, özellikler ve ilişkiler ifade edilebilir; quantifier'lar kullanılabilir. Örnek: $\forall X (\text{human}(X) \Rightarrow \text{mortal}(X))$. Çok daha fazla ifade gücü vardır ve logic programming'in temelini oluşturur.

S21. What is the difference between the universal and existential quantifiers in Prolog?

Cevap

Prolog'da değişkenler **evrensel olarak nicelendirilmiş** kabul edilir: bir kural veya fact'teki her değişken, o clause geçerli olduğu her değer için doğrudur.

Prolog, doğrudan **existential quantifier** içermez. Bunun yerine, “öyle bir X vardır ki” anlamı **sorgu yoluyla** verilir: `?- likes(X, mary).` — “Mary’yi seven bir X var mı?” sorusu aynı anlama gelir. Bir çözüm bulunursa varoluş ispatlanmış olur.

S22. Explain the difference between Prolog and a relational database system.

Cevap

- **Veri:** Her ikisi de fact şeklinde veri saklar. Ancak Prolog'da fact'ler parametrelili predicate'ler, veritabanında ise tablo satırlarıdır.
- **Çıkarım:** Prolog, *rule'larla* yeni bilgi türetebilir. İlişkisel veritabanları (SQL) sorgu yapabilir ama otomatik kural uygulaması yapmaz (Deductive database kavramı Prolog'a dayanır).
- **Arama:** Prolog depth-first + backtracking; SQL set-based (küme tabanlı) işlemler.
- **Özyineleme:** Prolog doğal olarak özyinelemeli ilişkileri tanımlayabilir; SQL'de özyineleme (WITH RECURSIVE) sonradan eklenmiştir.

S23. What is the box model (four port model) in Prolog?

Cevap

Dört port (box) modeli, Prolog'da her hedefin çalışmasını dört geçiş noktasıyla açıklayan trace modelidir:

- **CALL:** Hedef ilk kez çağrılıyor.
 - **EXIT:** Hedef başarıyla ispatlandı (unification ile).
 - **REDO:** Kullanıcı ; yazarak backtrack istedi; aynı hedef yeniden deniyor.
 - **FAIL:** Hedef ispat edilemedi; backtracking başlatılıyor.
- `trace.` komutuyla aktif edilir, `notrace.` ile kapatılır.

S24. How is Prolog's depth-first search strategy different from breadth-first search?

Cevap

Depth-first (DFS): Prolog'un kullandığı strateji. Bir dalı olabildiğince derine iner; başarısızlıkta geri döner. *Avantaj:* Az bellek kullanır (yalnızca aktif yol tutulur). *Dezavantaj:* Sonsuz dallarda takılabilir; en kısa çözümü bulamayabilir.

Breadth-first (BFS): Tüm dalları aynı derinlikte eş zamanlı keşfeder. *Avantaj:* En kısa çözümü bulur; sonsuz dalda takılmaz. *Dezavantaj:* Üstel bellek kullanımı. Prolog BFS kullanmaz çünkü bellekte tutulacak arama durumu sayısı kontrolsüz büyür.

S25. Why is logic programming often used in artificial intelligence applications?

Cevap

Logic programming'ın AI uygulamalarına uygunluğunun birçok nedeni vardır:

- **Bilgi temsili:** Gerçek dünya bilgisi fact ve rule olarak doğal biçimde ifade edilebilir.
- **Otomatik çıkarım:** Sisteme olgular ve kurallar verince yeni bilgi *otomatik olarak* türetilir.
- **Uzman sistemler:** Kural tabanlı uzman sistem mantığı Prolog'un doğal çalışma biçimiyle örtüşür.
- **Doğal dil işleme:** Dilbilgisi kuralları doğrudan Prolog rule'u olarak yazılabilir (Definite Clause Grammars).

- **Symbolic reasoning:** Sayısal değil sembolik işleme Prolog'un güçlü yönüdür.

12 Problem Set — Soru Çözümleri

P1. Compare Prolog's logical basis with that of functional languages like Scheme.

Cevap

Prolog (logic paradigm): Predicate calculus ve Horn clause temelli. Program = bilgi tabanı (fact + rule). Yürütme = otomatik teorem ispat (resolution + backtracking). “Ne doğrudur?” sorusunu yanıtlar. Kontrol akışı sisteme bırakılır.

Scheme (functional paradigm): Lambda calculus temelli. Program = fonksiyon tanımları. Yürütme = fonksiyon uygulamalarını değerlendirme. “Ne hesaplanır?” sorusunu yanıtlar. Kontrol akışı özyineleme ile programcı tarafından yönetilir.

Ortak nokta: Her ikisi de referential transparency hedefler; değişken/state kavramı merkezi değildir. Her ikisi de von Neumann imperatif modelinden uzaklaşmaya çalışır.

P2. Give an example of a query that would be problematic under the closed-world assumption.

Cevap

Örnek veri tabanı: `citizen(john, usa).`, `citizen(mary, uk).`

Sorgu: `?- citizen(ali, turkey).`

Cevap: `no` — Prolog bu bilgiyi bilmediği için “kesinlikle yanlış” der.

Sorun: Gerçek dünyada Ali'nin Türk vatandaşı olup olmadığını *bilmiyoruz*; bilgi tabanımız eksik. Kapalı dünya varsayımı bu belirsizliği “yanlış” olarak kodlar. **Open-world assumption (açık dünya varsayımı)** kullanan sistemler (örn. OWL/Description Logic) bu sorunu “bilinmiyor” cevabıyla ele alır.

P3. Explain why Prolog's search is incomplete and give a concrete example.

Cevap

Prolog'un depth-first araması, sonsuz bir dal varsa **bitmez**; dolayısıyla tüm çözümleri bulamaz.

Örnek — yanlış sıralı özyinelemeli kural:

```
1 % Yanlış sıralama - recursive case once:
2 ancestor(X, Y) :- ancestor(X, Z), parent(Z, Y). % Kural 1
3 ancestor(X, Y) :- parent(X, Y). % Kural 2
```

`?- ancestor(tom, ann).` sorgusu: Prolog Kural 1'i dener, `ancestor(tom, Z)` hedefini üretir, tekrar Kural 1'e gider... **sonsuz döngü**. Kural 2 hiç denenmez.

Doğru sıralama: base case (Kural 2) önce yazılmalı.

P4. Explain the difference between `=`, `==`, `is`, `:=` in Prolog.

Cevap

- `=`: Unification — değişken bağlaması yapabilir. `X = 3+5`: `X = +(3,5)` (hesaplamaz).
- `==`: Structural equality — iki term şu anda yapısal olarak eşit mi? Bağlama yapmaz. Serbest değişken kendisiyle eşittir ama başka şeyle eşit değildir.
- `is`: Sağ tarafı aritmetik olarak hesaplar, sol tarafla birleştirir. `X is 3+5`: `X = 8`.
- `:=`: Her iki tarafı da aritmetik olarak hesaplar ve sonuçları karşılaştırır. `3+5 := 2*4`: `true`.

P5. Why is Prolog's arithmetic evaluation different from that of Scheme?

Cevap

Scheme'de aritmetik ifadeler *değerlendirme sırasında* otomatik hesaplanır: $(+ 3 5)$ her zaman 8 döndürür çünkü Scheme işlevci bir dildir ve her ifadeyi değerlendirme (evaluate) temel modelidir.

Prolog'da ise $3 + 5$ bir yapısal term'dir — $+(3,5)$. Prolog **yalnızca** `is` operatörüyle açıkça istendiğinde aritmetik değerlendirme yapar. Bunun nedeni: Prolog'un temel işlemi *unification*'dır (hesaplama değil). $X = 3 + 5$ demek "X'i 3+5 yapısına eşitle" demektir, "X'i 8 yap" değil.

P6. What is the relationship between a Prolog database and a relational database?

Cevap

Prolog veri tabanı ile ilişkisel veri tabanı arasındaki ilişki: Her ikisi de **ilişkisel (relational) model** kullanır — veriler argümanlı predicate'ler (Prolog) veya satırlar (SQL) biçimindedir. Prolog öncülüdür; ilişkisel veritabanlarına benzer sorgulama gücü sağlar.

Fark: Prolog **kurallarla çıkarım** yapabilir (deductive database); ilişkisel veritabanları yalnızca sorguladığı verileri döndürür, yeni bilgi türetmez (SQL'de WITH RECURSIVE sonradan eklendi). Prolog'un arama mekanizması (backtracking) aynı zamanda mantıksal kapsamayı da sağlar.

P7. Discuss the advantages and disadvantages of depth-first search in Prolog.

Cevap

Avantajlar:

- *Bellek verimliliği:* Yalnızca aktif çözüm yolu bellekte tutulur; BFS'nin üstel bellek kullanımı yoktur.
- *Basitlik:* Implementasyonu kolaydır; stack tabanlı mekanizma doğrudan çalışır.
- *Hız:* İlk çözüme ulaşmak genellikle çok hızlıdır.

Dezavantajlar:

- *Tamamlanabilirlik yok:* Sonsuz bir dal varsa Prolog orada takılır.
- *Optimum değil:* En kısa çözümü bulamayabilir.
- *Sıra bağımlılığı:* Kural yazılış sırası hem doğruluğu hem sonlanmayı etkiler.

P8. Explain what happens when Prolog encounters a query with multiple solutions.

Cevap

Prolog bir hedefe ilk çözümü bulduğunda durur ve sonucu gösterir. Kullanıcı ; (noktalı virgül) yazarsa, Prolog **backtrack** ederek bir sonraki çözümü arar. Tüm çözümler tükenirse **false** yazar. `findall/3` yerleşik predicate'yle tüm çözümler aynı anda toplanabilir. Birden fazla çözümün olması Prolog'un temel güçlerinden biridir — tek sorguyla tüm olası cevaplar listelenebilir.

P9. What is the significance of the Prolog implementation by David Warren?

Cevap

David Warren, 1977'de Edinburgh Üniversitesi'nde **DEC-10 Prolog**'u geliştirdi. Bu implementasyon iki açıdan tarihi öneme sahiptir: (1) Tüm Prolog diyalektleri için *de facto* standart haline geldi; bugünkü ISO Prolog'un temeli bu implementasyondur. (2) Warren, Prolog çalıştırmak için soyut bir sanal makine tanımladı: **Warren Abstract Machine**

(WAM). WAM bugün hâlâ Prolog derleyicilerinin teorik temelini oluşturur ve Prolog’un verimli biçimde derlenmesini mümkün kılar.

P10. Compare the expressive power of logic programming with functional programming.

Cevap

Logic programming, ilişkileri hem *sorgulama* hem *üretme* hem de *doğrulama* yönünde kullanabilir. Tek bir predicate hem “X, Y’nin ebeveyni midir?” hem “Y’nin tüm çocukları kimdir?” sorularını yanıtlar. Bu *çift yönlülük (reversibility)* fonksiyonel dillerden üstün özelliğidir.

Fonksiyonel programming, hesaplamanın belirli yönünü (girdi → çıktı) açıkça tanımlar; tersini otomatik üretmez.

Pratik güç bakımından ise fonksiyonel diller (özellikle tip sistemleriyle) büyük ölçekli yazılım için daha uygundur; logic programming AI ve bilgi temsili alanında daha güçlüdür.

13 Programming Exercises — Çözümler

Listing 11: PE1 – Aile ilişkileri bilgi tabanı

```

1  % Temel olgular
2  parent(tom, bob).
3  parent(tom, liz).
4  parent(bob, ann).
5  parent(bob, pat).
6  parent(pat, jim).
7
8  male(tom). male(bob). male(pat). male(jim).
9  female(liz). female(ann).
10
11 % Tanımlanan ilişkiler
12 father(X, Y) :- parent(X, Y), male(X).
13 mother(X, Y) :- parent(X, Y), female(X).
14
15 sibling(X, Y) :-
16     parent(Z, X), parent(Z, Y), X \= Y.
17
18 grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
19 grandfather(X, Z) :- grandparent(X, Z), male(X).
20 grandmother(X, Z) :- grandparent(X, Z), female(X).
21
22 ancestor(X, Y) :- parent(X, Y).
23 ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
24
25 % Sorgular
26 % ?- father(tom, X).           % X = bob ; X = liz
27 % ?- grandmother(X, ann).      % no (tom buyukbaba, kadin buyukanne yok
    %   bu veri tabanında)
28 % ?- ancestor(tom, jim).       % yes (tom->bob->pat->jim)

```

Listing 12: PE2 – Matematik: En Büyük Ortak Bolen (GCD)

```

1  % Oklid algoritmasi ile EBOB
2  gcd(X, 0, X) :- X > 0.
3  gcd(X, Y, G) :-
4      Y > 0,
5      R is X mod Y,

```

```

6      gcd(Y, R, G).
7
8      % En Kucuk Ortak Kat (LCM) - EKOK
9      lcm(X, Y, L) :-
10         gcd(X, Y, G),
11         L is (X * Y) // G.
12
13     % Sorgular
14     % ?- gcd(48, 18, G).      % G = 6
15     % ?- lcm(4, 6, L).       % L = 12

```

Listing 13: PE3 – Sayisal ozellikleri test eden kurallar

```

1      % Pozitif/Negatif/Sifir
2      positive(X) :- X > 0.
3      negative(X) :- X < 0.
4      zero(X)      :- X == 0.
5
6      % Tek/Cift
7      odd(N)       :- N mod 2 =\= 0.
8      even(N)      :- N mod 2 == 0.
9
10     % Sayinin isaretini bul
11     sign(X, positive) :- X > 0.
12     sign(X, negative) :- X < 0.
13     sign(X, zero)     :- X == 0.
14
15     % Mutlak deger
16     absolute_value(X, X) :- X >= 0.
17     absolute_value(X, Y) :- X < 0, Y is -X.
18
19     % Iki sayinin arasindaki daha buyuk olan
20     greater(X, Y, X) :- X >= Y.
21     greater(X, Y, Y) :- Y > X.
22
23     % Sorgular
24     % ?- sign(-5, S).      % S = negative
25     % ?- absolute_value(-8, A). % A = 8
26     % ?- greater(7, 3, M). % M = 7

```

Listing 14: PE4 – Ozinelemeli sayisal hesaplamalar

```

1      % N-faktoryel
2      factorial(0, 1).
3      factorial(N, F) :-
4         N > 0,
5         N1 is N - 1,
6         factorial(N1, F1),
7         F is N * F1.
8
9      % Fibonacci dizisi
10     fib(0, 0).
11     fib(1, 1).
12     fib(N, F) :-
13         N > 1,
14         N1 is N - 1, N2 is N - 2,
15         fib(N1, F1), fib(N2, F2),
16         F is F1 + F2.
17
18     % Kuvvet: base^exp

```

```

19 power(_, 0, 1).
20 power(Base, Exp, Result) :-
21     Exp > 0,
22     Exp1 is Exp - 1,
23     power(Base, Exp1, R1),
24     Result is Base * R1.
25
26 % N'den 1'e toplam
27 sum_to(0, 0).
28 sum_to(N, S) :-
29     N > 0,
30     N1 is N - 1,
31     sum_to(N1, S1),
32     S is S1 + N.
33
34 % Sorgular
35 % ?- factorial(7, F).    % F = 5040
36 % ?- fib(10, F).        % F = 55
37 % ?- power(2, 8, P).    % P = 256
38 % ?- sum_to(100, S).    % S = 5050

```

Listing 15: PE5 – Bilgi tabani ve çok kosullu sorgulama

```

1  % Kurs bilgi tabani
2  course(cs101, programming, 4).    % kurs_adi, alan, kredi
3  course(cs201, data_structures, 3).
4  course(cs301, algorithms, 3).
5  course(cs341, prog_languages, 3).
6  course(math101, calculus, 4).
7
8  % Ogrenci kayıt bilgisi
9  enrolled(alice, cs101).
10 enrolled(alice, cs341).
11 enrolled(bob, cs101).
12 enrolled(bob, cs201).
13 enrolled(carol, cs341).
14 enrolled(carol, math101).
15
16 % Not bilgisi
17 grade(alice, cs101, 90).
18 grade(alice, cs341, 85).
19 grade(bob, cs101, 75).
20 grade(bob, cs201, 88).
21
22 % Kural: Bir alan kursunu aliyor mu?
23 takes_field(Student, Field) :-
24     enrolled(Student, Course),
25     course(Course, Field, _).
26
27 % Kural: Gecen mi?
28 passed(Student, Course) :-
29     grade(Student, Course, G),
30     G >= 60.
31
32 % Kural: CS alanında kayıtlı öğrenci
33 cs_student(Student) :-
34     takes_field(Student, _),
35     enrolled(Student, _).
36
37 % Sorgular

```

```

38 % ?- takes_field(alice, prog_languages). % yes
39 % ?- passed(bob, cs101). % yes (75>=60)
40 % ?- enrolled(carol, X). % X=cs341 ; X=math101

```

Listing 16: PE6 – Aritmetik ozet: kesirsayi hesabi

```

1 % Kesir tanimi: fraction(Pay, Payda)
2 % Sadelestirme
3 simplify(fraction(N, D), fraction(SN, SD)) :-
4     G is gcd_val(N, D),
5     SN is N // G,
6     SD is D // G.
7
8 gcd_val(X, 0, X) :- X > 0, !.
9 gcd_val(X, Y, G) :- Y > 0, R is X mod Y, gcd_val(Y, R, G).
10
11 % Toplama: a/b + c/d = (a*d + c*b) / (b*d)
12 add_fractions(fraction(N1,D1), fraction(N2,D2), Result) :-
13     N is N1*D2 + N2*D1,
14     D is D1 * D2,
15     G is abs(gcd_helper(N, D)),
16     RN is N // G,
17     RD is D // G,
18     Result = fraction(RN, RD).
19
20 gcd_helper(X, 0) :- X.
21 gcd_helper(X, Y) :- Y > 0, R is X mod Y, gcd_helper(Y, R).
22
23 % Daha temiz versiyon - is kullanarak
24 frac_add(N1/D1, N2/D2, RN/RD) :-
25     N is N1*D2 + N2*D1,
26     D is D1*D2,
27     gcd(N, D, G),
28     RN is N // G,
29     RD is D // G.
30
31 % Sorgular
32 % ?- frac_add(1/3, 1/6, R). % R = 1/2
33 % ?- frac_add(2/5, 3/10, R). % R = 7/10

```

Listing 17: PE7 – Kapsamli sorgu ornekleri ve trace analizi

```

1 % --- Tam soru-cevap seti ---
2
3 % Veritabani
4 parent(tom, bob). parent(tom, liz).
5 parent(bob, ann). parent(bob, pat).
6
7 ancestor(X,Y) :- parent(X,Y).
8 ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
9
10 % SORGU 1: Direkt ata?
11 % ?- ancestor(tom, ann).
12 % Adim 1: ancestor(tom,ann) <- kural1: parent(tom,ann)? FAIL
13 % Adim 2: ancestor(tom,ann) <- kural2: parent(tom,Z), ancestor(Z,ann)
14 % Z=bob: parent(tom,bob) OK
15 % ancestor(bob,ann) <- kural1: parent(bob,ann) OK => YES
16
17 % SORGU 2: Tom'un torunlari?
18 % ?- ancestor(tom, X).

```

```
19 %    X = bob (parent(tom,bob))
20 %    X = liz (parent(tom,liz))
21 %    X = ann (tom->bob->ann)
22 %    X = pat (tom->bob->pat)
23
24 % SORGU 3: Toplam kac torun?
25 % ?- findall(X, ancestor(tom,X), L), length(L, N).
26 %    L = [bob, liz, ann, pat], N = 4
27
28 % SORGU 4: Ann'in atalari?
29 % ?- ancestor(X, ann).
30 %    X = bob (parent(bob,ann))
31 %    X = tom (tom->bob->ann)
32
33 % NOT: trace ile SORGU 1'i inceleyin:
34 % ?- trace, ancestor(tom, ann).
```